

dbt & Data Modeling

SQL Transformations with Version Control and Testing

■ DBT

■ LEARNING OUTCOME

By the end of this module you will build a complete dbt project with staging, intermediate, and mart models, apply ref() and source(), write dbt tests, generate documentation, and use macros and incremental models.

■ Skills You Will Gain

- Understand dbt's role in the modern ELT data stack
- Build dbt models using SELECT statements with ref() and source()
- Apply dbt tests: not_null, unique, accepted_values, relationships
- Write reusable Jinja macros for DRY SQL
- Configure incremental models to process only new data
- Generate and serve interactive data documentation

■ dbt is the #1 skill in the modern data stack. Every company using Snowflake or BigQuery is adopting dbt. It can add ■4–8 LPA to your salary immediately.

SECTION 1

What is dbt and Why Does It Matter?

dbt = data build tool

dbt lets you write SELECT statements — dbt handles the CREATE TABLE / VIEW automatically. Key innovations: Version control: all transforms in Git — review, rollback, branching like software Testing: define data quality rules that run on every deploy Documentation: auto-generate searchable docs with column descriptions and lineage Lineage: visualise how every table depends on others — debug data issues fast Jinja templating: macros, variables, conditional logic in SQL dbt runs INSIDE your warehouse compute — no separate transformation server needed.

SECTION 2

dbt Project Structure

```
my_dbt_project/ dbt_project.yml # project name, version, model configs profiles.yml # database
connection (local, not committed) models/ staging/ # stg_* models: clean source data 1:1
stg_orders.sql stg_customers.sql schema.yml # tests + descriptions for staging models sources.yml #
declare raw source tables intermediate/ # int_* models: business logic, joins
int_order_items_enriched.sql marts/ # fct_* and dim_* final models fct_orders.sql dim_customers.sql
schema.yml macros/ # reusable Jinja functions generate_surrogate_key.sql clean_text.sql seeds/ #
static CSV reference data country_codes.csv snapshots/ # SCD Type 2 history tracking
customers_snapshot.sql tests/ # custom singular tests assert_revenue_positive.sql
```

SECTION 3

dbt Models

```
-- models/staging/stg_orders.sql -- Staging: clean source data, rename columns, cast types {{
config(materialized='view') }} WITH source AS ( -- source() function: reference raw table from
sources.yml SELECT * FROM {{ source('raw', 'orders') }} ), cleaned AS ( SELECT order_id AS
order_id, customer_id AS customer_id, CAST(order_date AS DATE) AS order_date, ROUND(CAST(amount AS
FLOAT64), 2) AS amount, LOWER(TRIM(status)) AS status, COALESCE(discount_pct, 0) AS discount_pct,
CURRENT_TIMESTAMP() AS _loaded_at FROM source WHERE order_id IS NOT NULL AND amount > 0 ) SELECT *
FROM cleaned -- models/marts/fct_orders.sql -- Mart: join staging models, calculate metrics {{
config(materialized='table') }} WITH orders AS ( -- ref() function: reference another dbt model
SELECT * FROM {{ ref('stg_orders') }} ), customers AS ( SELECT * FROM {{ ref('stg_customers') }} ),
final AS ( SELECT o.order_id, o.customer_id, o.order_date, o.amount, o.amount * (1 -
o.discount_pct) AS net_revenue, c.name AS customer_name, c.segment AS customer_segment, c.city AS
customer_city FROM orders o LEFT JOIN customers c USING (customer_id) ) SELECT * FROM final
```

SECTION 4

dbt Tests and Schema YAML

```
# models/staging/schema.yml version: 2 sources: - name: raw schema: raw_data tables: - name: orders
description: 'Raw orders from transactional database' columns: - name: order_id description:
'Unique order identifier from source system' models: - name: stg_orders description: 'Cleaned and
typed orders from raw source' columns: - name: order_id description: 'Unique order identifier'
tests: - not_null - unique - name: status tests: - accepted_values: values:
['pending','processing','shipped','delivered','cancelled'] - name: customer_id tests: - not_null -
relationships: to: ref('stg_customers') field: customer_id - name: amount tests: - not_null -
dbt_utils.expression_is_true: expression: '>= 0'
```

dbt Commands Reference

Command	What It Does
dbt run	Build all models — create tables/views in warehouse
dbt run --select stg_orders	Build only the stg_orders model
dbt run --select staging.*	Build all models in staging/ folder
dbt run --select +fct_orders	Build fct_orders AND all its upstream dependencies
dbt test	Run all schema and singular tests
dbt docs generate	Generate documentation site from schema.yml and code
dbt docs serve	Open interactive documentation in browser at localhost:8080
dbt snapshot	Run SCD Type 2 snapshot models
dbt seed	Load CSV seed files as tables into warehouse
dbt debug	Test database connection from profiles.yml
dbt compile	Compile Jinja templates to SQL without running
dbt source freshness	Check if source tables were updated within SLA

■ PRACTICE Q & A

Q1. What does `{{ ref('stg_orders') }}` do in a dbt model?

A: Creates a dependency: dbt knows this model depends on stg_orders and builds it first. dbt compiles it to the full warehouse path (schema.stg_orders) automatically. Powers the lineage graph.

Q2. What is the difference between `materialized='view'` and `materialized='table'`?

A: View: virtual — SQL runs fresh every query, no storage, always up-to-date, slow for complex queries. Table: physical — pre-computed and stored, fast queries, stale until next dbt run.

Q3. What is an incremental materialisation and when should you use it?

A: Processes only new/changed rows since last run using: `WHERE updated_at > (SELECT MAX(updated_at) FROM {{ this }})`. Use for large fact tables where full rebuild takes hours.

Q4. What is the purpose of schema.yml in dbt?

A: Declares: model and column descriptions (used in docs), data quality tests (not_null, unique, accepted_values, relationships), and source definitions. dbt test reads this and runs checks automatically.

Q5. What is a dbt macro and how does it improve code quality?

A: Reusable Jinja function that generates SQL. E.g. `generate_surrogate_key(['order_id','product_id'])` creates a consistent hash key. DRY principle — write once, use in every model that needs it.

■ dbt Cheat Sheet	
dbt run	Build all models
dbt test	Run all data quality tests
dbt docs generate && serve	Build and open documentation
{{ ref('model_name') }}	Reference another dbt model
{{ source('schema', 'table') }}	Reference raw source table

<code>materialized='view'</code>	Virtual – no storage
<code>materialized='table'</code>	Physical – stored in warehouse
<code>materialized='incremental'</code>	Process only new rows each run
<code>test: not_null</code>	Column cannot have NULL values
<code>test: unique</code>	No duplicate values in column
<code>test: accepted_values: [...]</code>	Only these values are allowed
<code>dbt_utils.surrogate_key([...])</code>	Generate MD5 surrogate key